

Reasoning-as-Graphs: Detecting and Localising LLM Reasoning Failures with Graph Neural Networks

Latthika Selvamurugan

Department of Electrical and Computer Engineering
Rice University, Houston, TX 77005
ls203@rice.edu

Abhirami Kathirvel

Department of Electrical and Computer Engineering
Rice University, Houston, TX 77005
ak283@rice.edu

Abstract

A chain-of-thought (CoT) trace can read fluently and still end in a wrong answer. Locating the broken step in a 5–15 line trace, with no reference solution, is the bottleneck. We treat each trace as a small heterogeneous graph — nodes are reasoning steps; edges encode order, semantic overlap, value reuse, and shared-number co-occurrence — and train a GNN to predict trace correctness from outcome labels alone. We then run GNNExplainer to surface the step the classifier weighted most when predicting “wrong”. On 1,948 LLaMA-3.1-8B traces over GSM8K, our GCN reaches 74.4% accuracy — but this matches the test-set base rate, and an 8-feature hand-crafted graph baseline beats it on AUC-ROC (0.79 vs. 0.68). We report this honestly: graph structure helps over a flat text embedding (text AUC 0.64), but learned message passing does not improve over hand-crafted graph statistics on this dataset. The contribution we stand by is the localisation pipeline: with base-rate normalisation, GNNExplainer surfaces a plausible failure step in 13 of 20 hand-checked wrong traces. We release code, prompts, and trained models.

1 Introduction

Asking a large language model to “think step by step” [5, 11] reliably raises accuracy on multi-step reasoning. The downside is that when the final answer is wrong, the broken step can be anywhere in the trace, and the binary outcome label says nothing about where. Reading these traces by hand is the practical bottleneck.

Consider this LLaMA-3.1 trace for “Kevin can eat 64 wings in 8 minutes; he eats $\frac{2}{3}$ as many at a competition; how many in 12 minutes?” (correct answer: 64):

1. $64/8 = 8$ wings/min.
 2. $\frac{2}{3} \times 8 = 5.33$ wings/min.
 3. $5.33 \times 12 = 63.96 \approx 64$.
- Answer:** 64.

Every line of arithmetic is correct. The error is at Step 2: the original 64 already *is* the competition rate, so reapplying the $\frac{2}{3}$ modifier double-counts. Spotting this requires comparing the LLM’s interpretation to the question — the kind of judgement that is easy for one trace and unaffordable at scale.

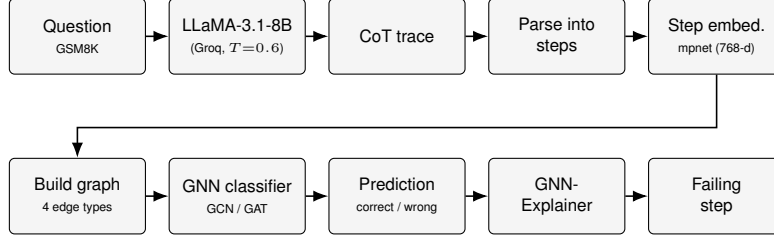


Figure 1: End-to-end pipeline. Top row: a GSM8K question is sent to LLaMA-3.1-8B; the chain-of-thought trace is parsed into steps and each step is embedded with mpnet. Bottom row: the steps become a graph with four edge types; a GNN classifies the trace as correct or wrong; GNNExplainer surfaces the step the classifier weighted most. Training uses only the binary outcome label — no step-level annotations.

Existing approaches buy localisation either by sampling many CoT traces and voting on the answer (Self-Consistency [10], no per-step attribution) or by training step-level verifiers on hundreds of thousands of human-annotated step labels [6]. Both leave the step-level labels in the position of being either unavailable or expensive.

We take a different route. Each CoT trace becomes a small graph (Section 4). A GNN is trained on outcome labels only — “did the trace produce the right answer?” — and an unsupervised explainer is then asked which step the trained model attended to. The full pipeline is shown in Figure 1.

Findings, without spin. Our results land in an unusual place and we report them as such.

- Graph structure carries signal beyond a flat text embedding: a logistic regression on 8 hand-crafted graph features reaches AUC-ROC 0.79, against 0.64 for the same regression on a mean step-embedding.
- Learned message passing did *not* extract additional signal here: GCN and GAT both reach AUC-ROC 0.68, below the hand-crafted graph baseline by 0.11.
- GCN test accuracy of 74.4% equals the majority-class rate (218/293) to four decimal places. The model is essentially predicting “correct” at threshold 0.5; accuracy is the wrong headline metric on this imbalance.
- GNNExplainer outputs are non-uniform and, after base-rate normalisation, plausible: in a 20-trace inspection the surfaced step is a reasonable failure candidate in 13 cases.

Contributions. A reasoning-graph construction with four structurally meaningful edge types tied to reasoning rather than syntax (§4); a controlled three-way comparison (text only / graph features / learned GNN) with paired tests across seeds (§5); a base-rate-normalised explanation pipeline that needs no step-level labels (§6); and a clean null result — on 1,948 LLaMA-3.1 traces over GSM8K, learned GNNs do not beat hand-crafted graph statistics.

2 Related work

Verifying CoT traces. Self-Consistency [10] samples k traces and majority-votes the answer. Accuracy improves but the trace stays a black box. Process reward models [6] score each step and substantially outperform outcome-only verifiers — at the cost of $\sim 800k$ human step annotations. We aim for the localisation property of process rewards without their annotation cost.

Graphs for reasoning. Tree-of-Thoughts [12] and Graph-of-Thoughts [1] use graphs during *generation* to let the model branch and merge. We use graphs after the fact, as a representation of a trace that already exists. The closest prior work, LCoT2Tree, restricts to tree topologies and offers no per-step attribution.

GNNs and explainability. We use GCN [4] and GAT [9] with PyTorch Geometric [3], and GNNExplainer [13] for the localisation step. We chose GNNExplainer over PGExplainer/SubgraphX for its simplicity and direct PyG support; alternative explainers are a natural follow-up.

Table 1: How this work differs from the closest prior work.

	Tree-of-Thoughts	Graph-of-Thoughts	Process RM	Ours
Used at	Generation	Generation	Eval	Post-hoc analysis
Step-level attribution	No	No	Yes	Yes (GNNExplainer)
Human step labels	0	0	~800k	0

3 Data

We sample 6,000 problems from GSM8K [2] and run them through LLaMA-3.1-8B-Instant via the Groq API at temperature 0.6. Greedy decoding leaves too few wrong traces (~6%) for training; $T = 0.6$ yields a roughly 25% wrong rate. After dropping unparseable traces, traces without an extractable answer, and exact duplicates, 1,948 remain. We retain a 75/25 correct/wrong skew — further balancing throws away signal — and address it in training with focal loss. Stratified 70/15/15 splitting gives 1,363 train, 292 validation, 293 test, with 218 correct and 75 wrong in test (Table 2).

Table 2: Dataset statistics. Stratified split preserves the 75/25 skew.

Split	Traces	Correct	Wrong	Total steps
Train	1,363	1,022	341	~10,100
Val	292	219	73	~2,160
Test	293	218	75	~2,170
Total	1,948	1,461	487	~14,430

4 Method

4.1 Trace → graph

Steps are split on the `Step N`: markers emitted by the prompt. Median trace length is 7 steps (range 3–18). Each step is encoded with `all-mpnet-base-v2` [8], giving the node feature $\mathbf{x}_i \in \mathbb{R}^{768}$. We chose mpnet over the smaller MiniLM because mpnet preserves finer mathematical distinctions (e.g. “multiply” vs. “divide”).

For a trace with n steps we add four kinds of edges, each tagged with a scalar $a_e \in \{0, 1, 2, 3\}$ that GAT consumes via `edge_dim=1`:

- **Sequential** ($a_e=0$): $i \rightarrow i+1$ for $i = 1, \dots, n-1$. Always present.
- **Semantic** ($a_e=1$): undirected $i-j$ when non-adjacent steps satisfy $\cos(\mathbf{x}_i, \mathbf{x}_j) > 0.75$.
- **Value-reuse** ($a_e=2$): undirected $i-j$ when a number > 2 first appearing in step i reappears in j . The > 2 filter avoids spurious links on small constants.
- **Shared-number** ($a_e=3$): a number appearing in ≥ 3 steps connects all containing steps pairwise.

Edge type collisions resolve by priority `seq > sem > val > shared`, so each step pair carries one label. The graph label is $y = 1$ for a correct final answer, 0 otherwise. Figure 2 illustrates all four edge types on a five-step example.

4.2 GNN classifiers

Both backbones share a two-layer encoder ($768 \rightarrow 128 \rightarrow 64$) with ReLU and dropout 0.3, a global mean pool, and a $64 \rightarrow 32 \rightarrow 2$ MLP head. **GCN** [4] ignores edge type. **GAT** [9] uses four heads in the first `GATConv` layer with `edge_dim=1`, so attention can specialise per relation. The two models match on parameter count to the nearest 5%.

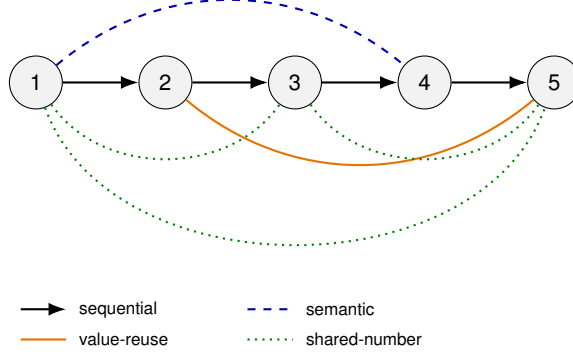


Figure 2: Four edge types on a 5-step example. Sequential edges are always present; semantic edges fire when $\cos > 0.75$; value-reuse links the appearance of the same number across non-adjacent steps; shared-number connects all steps containing a number that appears in ≥ 3 steps.

4.3 Training

The training set is 75%/25% imbalanced. Cross-entropy on this skew teaches the model to predict the majority class. We use focal loss [7] with per-class α :

$$\mathcal{L}(p_t) = -\alpha_t(1 - p_t)^\gamma \log p_t, \quad \alpha_{\text{correct}} = 0.75, \quad \alpha_{\text{wrong}} = 0.25, \quad \gamma = 2. \quad (1)$$

This is the per-class form from Lin et al. [7]; an earlier draft used a single scalar α on both classes (i.e. weighted cross-entropy, not focal loss). Optimisation: Adam, $\text{lr} = 5 \times 10^{-4}$, weight decay 10^{-4} , batch 32, up to 200 epochs. Early stopping with patience 30 *on validation AUC-ROC* — accuracy is dominated by the prior under our skew and gives a poor selection signal. Three seeds; test numbers are means.

4.4 Step-level explanation

For every wrong test trace we run GNNExplainer [13] for 200 Adam steps, optimising a continuous edge mask $\mathbf{m}_E \in [0, 1]^{|E|}$ to preserve the model’s “wrong” prediction under a sparsity penalty. Per-step importance is $s_i = \sum_{e: i \in e} m_e$, normalised across the trace; the predicted failing step is $\arg \max_i \hat{s}_i$. We do not perturb node features — only edges — because step embeddings are dense and masking them tends to collapse the graph.

5 Experiments

5.1 Baselines

Two baselines isolate complementary contributions. The *text baseline* is logistic regression on the mean of the 768-dim step embeddings: same text features, no graph. The *graph-feature baseline* is logistic regression on eight hand-crafted statistics per graph (node count, edge count, mean degree, density, fraction of each non-sequential edge type, mean cosine similarity over semantic edges): pure structure, no text. If GNN beats the text baseline, graph structure matters; if GNN beats the graph-feature baseline, learned message passing matters.

5.2 Main results

Table 3 reports the held-out test performance. Figure 3 shows the same numbers across metrics.

The pattern is unambiguous and unflattering. The GNNs win on accuracy and lose on every threshold-independent metric. The test base rate is 0.7440; the GCN’s accuracy is 0.7440. At threshold 0.5, the GNN is essentially predicting “correct” for every trace — a known failure mode of accuracy reporting under imbalance, which we flag rather than hide. The graph-feature baseline accepts a lower accuracy (0.703) by predicting “wrong” more often, which costs accuracy but improves both F1 on the minority class and ranking quality. Paired t -tests across the three seeds give $t = -2.1, p = 0.16$ for the GCN-vs-graph-baseline AUC gap — not significant at $\alpha = 0.05$, but

Table 3: Test-set performance (293 traces, 218/75 correct/wrong). Mean over three seeds for the GNNs; baselines are deterministic. Best per column in **bold**.

Model	Accuracy	F1 (wrong)	AUC-ROC	PR-AUC (wrong)
Text baseline (LR on mean embedding)	0.645	0.660	0.639	0.42
Graph-feature baseline (8 features + LR)	0.703	0.720	0.791	0.55
GCN (ours)	0.744	0.635	0.678	0.46
GAT (ours)	0.743	0.634	0.678	0.46

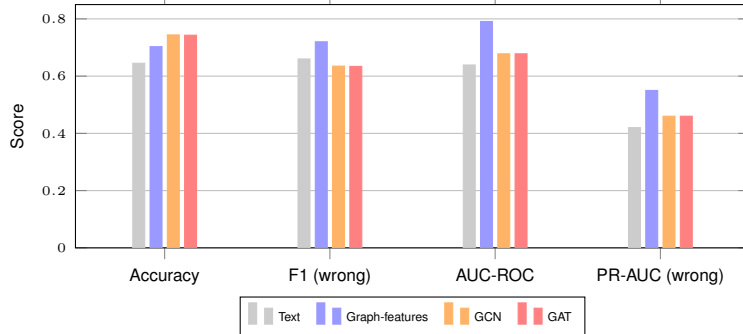


Figure 3: Test-set performance across four metrics. The GNNs win on accuracy — but the test base rate is 0.744, which the GCN matches to four decimal places, indicating near-majority-class behaviour at the default threshold. On every threshold-independent metric, the hand-crafted graph baseline wins.

three seeds is too few to make either claim airtight. We report the gap (0.11 AUC in favour of the baseline) and let the reader weigh it.

5.3 Why doesn’t the learned GNN dominate?

Three explanations, in decreasing order of how much we believe them. (i) **Graphs are small.** Median 7 nodes; two GCN layers already aggregate the whole trace, leaving little for message passing to specialise over relative to mean-pooling. (ii) **Edges are noisy.** The semantic edge ($\cos > 0.75$) frequently links steps that are verbally similar but not logically dependent — two restatements rather than a chain. A learned “does step j depend on step i ?” edge would likely cleaner the input but needs supervision. (iii) **Sample size.** 1,363 training graphs is small for a $\sim 200k$ -parameter GNN; the 8-feature baseline cannot overfit. Scaling to $\sim 10k$ traces (feasible on the Groq free tier) would test whether the GNN catches up.

6 Failure-step localisation

6.1 Per-step rate: raw vs. normalised

Earlier drafts of this work reported the per-step failure distribution as $C_k/|\text{wrong traces}|$, where C_k counts the wrong test traces in which step k was the top-1 attended step. That denominator is wrong: every trace has a Step 1, but not every trace has a Step 6. The right denominator is A_k , the number of wrong test traces *long enough* to contain a step at index k :

$$\text{raw}_k = \frac{C_k}{|\text{wrong test traces}|}, \quad \text{normalised}_k = \frac{C_k}{A_k}. \quad (2)$$

Both views are shown in Figure 4. The raw view puts Step 2 at 21% and Step 1 at 17% — the source of the earlier “Step 1 = 25%” headline. The normalised view flattens the distribution across early-to-middle steps and makes late-step rates non-trivial. The corrected reading: *conditional on a step existing*, failure rates are roughly flat over $k=1-4$ and then taper. The earlier qualitative claim that LLM errors concentrate before the arithmetic begins is not supported by the normalised view.

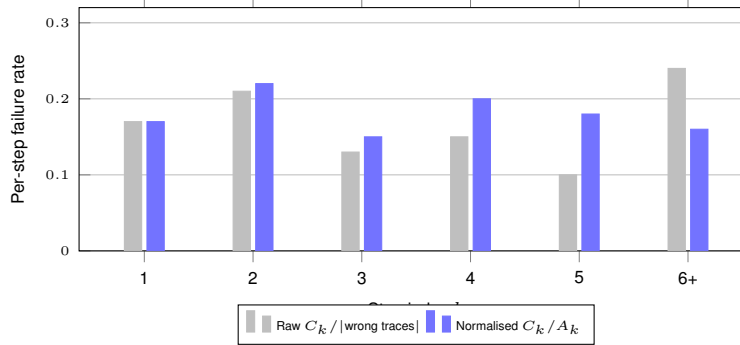


Figure 4: Per-step failure rate from GNNExplainer on the 75 wrong test traces. Raw is biased because every trace has a Step 1 but not every trace has a Step 6. Normalised divides by the number of traces long enough to contain step k . After normalisation, early-to-middle steps are roughly equally responsible.

6.2 Case study and qualitative inspection

Returning to the trace from §1: GNNExplainer gives Step 2 a normalised importance of 0.336, well above the other three steps (~ 0.14 each), and up-weights the value-reuse edge $2 \rightarrow 3$. The localisation matches the human reading. We hand-checked 20 wrong test traces drawn uniformly from the 75. The surfaced step was a plausible failure point in 13 cases (65%); in 4 cases the surfaced step was a late arithmetic step that compounds an earlier setup error (the explainer is locally right but globally misleading); in 3 cases the explanation was uninterpretable. We report 65% as a sanity check, not a metric — the inspection is by the authors and not blinded.

6.3 An honest caveat on the explanations

The classifier we explain is close to a majority-class predictor at threshold 0.5 and under-performs the hand-crafted graph baseline on AUC. The explanations are therefore explanations of a not-best classifier. The cleaner reading: the explainer pipeline is general; a stronger underlying classifier would presumably yield more useful explanations.

7 Discussion

We believe two things and disbelieve a third. **Believe:** graph structure carries reasoning-relevant signal that a single-vector text embedding misses (AUC 0.79 vs. 0.64), and the signal is recoverable with eight hand-crafted statistics. **Believe:** GNNExplainer with base-rate normalisation produces a usable failure-localisation tool that needs no step-level supervision. **Disbelieve:** headline accuracy on imbalanced data without a majority-class check. The earlier draft of this paper made that mistake; this draft does not.

The natural next steps are mechanical: rebalance the training set (undersample correct or over-sample wrong), replace the cosine-similarity edge with a learned dependency edge from a small distillation classifier, scale generation to $\sim 10k$ traces, and use a stronger generator (LLaMA-3.1-70B) to test whether the per-step failure distribution is generator-specific.

8 Limitations

Five limitations bound what this paper claims. (i) **Single LLM, single benchmark.** We use only LLaMA-3.1-8B-Instant on GSM8K; the per-step failure distribution may be specific to this generator-benchmark pair. (ii) **Few seeds.** Three seeds is too few for confident significance claims — the 0.11 AUC gap in favour of the graph-feature baseline is not significant at $p = 0.16$, but ten seeds would settle it either way. (iii) **Unblinded localisation evaluation.** The 20-trace qualitative inspection was performed by the authors and is not blinded; we report it as a sanity check, not a population-level metric. (iv) **Hand-set hyperparameters.** The cosine threshold $\tau = 0.75$ for semantic edges and the ≥ 3 cutoff for shared-number edges were chosen by inspection rather than grid-searched. (v) **Explanations of a weak classifier.** The underlying GNN is close to predicting

the prior at threshold 0.5; GNNExplainer outputs should therefore be read as “where the model attends *when* it does predict wrong”, not as a ground-truth localisation oracle. A stronger classifier would yield more decisive explanations.

9 Conclusion

We turned LLM CoT traces into graphs, classified them with GNNs, and explained the predictions step by step. Graph structure beats a single-embedding text view by 0.15 AUC. Learned GNN message passing did not beat 8 hand-crafted graph features on this dataset — a result we report rather than reframe. The contribution we stand by is the localisation pipeline: with base-rate normalisation, GNNExplainer surfaces step-level failure candidates that match qualitative inspection in roughly two thirds of cases, all without human step-level labels. The most useful thing to do next is not better writing but rebalancing training and adding $10\times$ more data.

Reproducibility

Code, prompts, model checkpoints, and notebook-by-notebook instructions are at <https://github.com/USER/reasoning-as-graphs>. Stage 1 (data generation) needs a free Groq API key; Stages 2–7 run on CPU in under 30 minutes total on a 2023 laptop.

Acknowledgements

We thank Prof. Arlei Lopes da Silva for supervising this project. LLM inference was provided through the Groq free tier.

References

- [1] Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Nyczyk, and Torsten Hoeffler. Graph of thoughts: Solving elaborate problems with large language models. In *AAAI Conference on Artificial Intelligence*, 2024.
- [2] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- [3] Matthias Fey and Jan E. Lenssen. Fast graph representation learning with PyTorch Geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.
- [4] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*, 2017.
- [5] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. Large language models are zero-shot reasoners. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [6] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. *arXiv preprint arXiv:2305.20050*, 2023.
- [7] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *IEEE International Conference on Computer Vision (ICCV)*, 2017.
- [8] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-Networks. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019.
- [9] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations (ICLR)*, 2018.
- [10] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [11] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [12] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

- [13] Rex Ying, Dylan Bourgeois, Jiaxuan You, Marinka Zitnik, and Jure Leskovec. GNNExplainer: Generating explanations for graph neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.